

Open in app ↗

Sign up

Sign in

Medium

Search

Write



Market Analysis with K-Means Clustering Algorithm: Identifying Support and Resistance Levels



Cemal Öztürk, Ph.D.

Follow

7 min read · Feb 8, 2024



1



Photo by Tötös Ádám on Unsplash

Introduction

In financial markets, investing requires a solid analysis and strategy to achieve profitable and sustainable returns. Technical analysis is a popular tool that helps investors predict future trends by studying the historical price movements of the market. One important technique used in this analysis is identifying **support and resistance levels**.

Support and resistance levels are imaginary lines that limit the upward or downward movement of the price. A **support level** is at which the price is expected to bounce before falling further. A **resistance level** is at which the price is expected to encounter resistance before rising further. Identifying these levels can help investors optimize their trading decisions and risk management.

The K-Means clustering algorithm is a powerful tool that can be used to identify support and resistance levels. This algorithm groups data into clusters with similar characteristics. The K-Means algorithm can automatically identify resistance levels in financial markets by analyzing historical price data and identifying clusters of data points with similar price movements.

What is the K-means Clustering Algorithm?

The K-means clustering algorithm is an unsupervised learning technique to partition a dataset into a predefined number of clusters or groups. This algorithm groups data points into clusters while computing the centroid of each cluster. The K-means algorithm performs clustering based on the similarity between data points.

Advantages of K-Means Algorithm

- Identifies support and resistance levels automatically and objectively.
- Can be used on different time frames and different asset classes.
- It is relatively easy to use and can be used by investors without technical analysis knowledge.

Disadvantages of K-Means Algorithm

- The choice of K value can significantly affect the accuracy of the results.
- The algorithm may not always accurately predict sudden and unexpected market fluctuations.
- Provides more reliable results when used in conjunction with other technical analysis tools.

Identifying Support and Resistance Levels Using K-Means Algorithm

1. **Data Collection:** Collect historical price data for different time frames (daily, weekly, monthly).
2. **Data Preprocessing:** Apply data preprocessing techniques to normalize the data and handle missing values.
3. **Selecting the K-Means Algorithm:** There are different variations of the K-Means algorithm. Choose the variation that best suits your data and analysis needs.
4. **Determining the K value:** K represents the number of clusters the algorithm will divide the data into. Experiment with different K values and select the one that yields the most optimal results.
5. **Running the Algorithm:** Run the K-Means algorithm with the selected K value.

6. Analyzing the Results: Analyze the clusters created by the algorithm and identify support and resistance levels.

Sample Code:

First, importing these libraries gives you access to various functionalities for data manipulation, analysis, clustering, and visualization, which are essential for many data science tasks.

```
# Import necessary libraries
import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.cluster import KMeans
from kneed import DataGenerator, KneeLocator
import matplotlib.pyplot as plt
```

This code block utilizes the `yfinance` library to download stock data for a specified date range, focusing on the daily closing prices of the stock. Here's the breakdown of the steps:

```
ticker = 'GOOG'          # Ticker symbol of the stock to trade (e.g., "GOOG"
start_date = '2018-03-22' # Start date for fetching the data

# Downloading stock data for the specified date range using the yfinance library
df = yf.download(
    tickers=ticker,          # Ticker symbol of the stock
    start=start_date,        # Start date for fetching data
    progress=False           # Whether to show download progress
)[['Open', 'High', 'Low', 'Close']] # Selecting only the 'Open', 'High', 'L
```

```
# Converting the 'Close' column from the DataFrame to a NumPy array
df = np.array(df['Close'])
```

The variable `df` holds an array of daily closing prices of the specified stock for the given date range. You can further analyze this data and develop trading strategies based on it.

Get Cemal Öztürk, Ph.D.'s stories in your inbox

Join Medium for free to get updates from this writer

Enter your email

Subscribe

This code block computes the sum of squared distances (inertia) for different values of `K` (number of clusters) using the `KMeans` algorithm. Then, it uses the `KneeLocator` from the `kneed` library to identify the elbow point in the curve. Here's the explanation of each line:

```
sum_of_squared_distances = [] # Initialize an empty list to store the sum of sq
K = range(1, 15)             # Define a range of K values from 1 to 14 (number

# Iterate over each value of K
for k in K:
    km = KMeans(n_clusters = k) # Create a KMeans object with th
    km = km.fit(df.reshape(-1, 1)) # Fit the KMeans model to the da
    sum_of_squared_distances.append(km.inertia_) # Append the sum of squared di

# Use KneeLocator to find the elbow point in the curve
kn = KneeLocator(K, sum_of_squared_distances, S = 1.0, curve = "convex", directi
kn.plot_knee() # Plot the knee point
```

This code helps determine the optimal number of clusters (K) for KMeans clustering by identifying the elbow point, which indicates where the rate of decrease in inertia sharply changes.

```
kmeans = KMeans(n_clusters=kn.knee).fit(df.values.reshape(-1,1))
c = kmeans.predict(df.values.reshape(-1,1))
minmax = []
for i in range(kn.knee):
    minmax.append([-np.inf,np.inf])
for i in range(len(df)):
    cluster = c[i]
    if df.values[i] > minmax[cluster][0]:
        minmax[cluster][0] = df.values[i]
    if df.values[i] < minmax[cluster][1]:
        minmax[cluster][1] = df.values[i]
```

- `kmeans = KMeans(n_clusters=kn.knee).fit(df.values.reshape(-1,1))` : K-means clustering algorithm is applied to the dataset with the optimal number of clusters (determined by `kn.knee`). The `fit()` method fits the K-means model to the data.
- `c = kmeans.predict(df.values.reshape(-1,1))` : The `predict()` method assigns each data point in the dataset to one of the clusters identified by the K-means algorithm.
- `minmax = []` : An empty list `minmax` is initialized to store each cluster's minimum and maximum values.
- `for i in range(kn.knee):` : A loop iterates over each cluster (the number of clusters determined by `kn.knee`).
- `minmax.append([-np.inf,np.inf])` : For each cluster, an initial range of $[-\infty, \infty]$ is appended to `minmax`, representing the initial minimum and maximum values.

- `for i in range(len(df)):` : Another loop iterates over each data point in the dataset.
- `cluster = c[i]` : The cluster assigned to the current data point is retrieved from the `c` array.
- `if df.values[i] > minmax[cluster][0]:` : If the value of the current data point is greater than the current maximum value for its assigned cluster,
- `minmax[cluster][0] = df.values[i]` : Update the maximum value for the cluster to the value of the current data point.
- `if df.values[i] < minmax[cluster][1]:` : If the value of the current data point is less than the current minimum value for its assigned cluster,
- `minmax[cluster][1] = df.values[i]` : Update the minimum value for the cluster to the value of the current data point.

This code segment creates a scatter plot to visualize the data points and highlight each cluster's minimum and maximum values. Let's break it down:

```
from matplotlib import pyplot as plt
for i in range(len(df)):
    colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
    c = kmeans.predict(df[i].reshape(-1,1))[0]
    color = colors[c]
    plt.scatter(i, df[i], c=color, s=1)
```

- Loops through each data point in the dataset.
- Defines a list of colors (`colors`) to use for plotting.

- Predicts the cluster label for the current data point using the trained K-means model (`kmeans.predict(df[i].reshape(-1,1))[0]`).
- Retrieves the corresponding color for the cluster label.
- Plots a scatter point with x-coordinate `i` , y-coordinate `df[i]` , color `color` , and size `s=1` .

```
for i in range(len(minmax)):
    plt.hlines(minmax[i][0], xmin=0, xmax=len(df), colors='g')
    plt.hlines(minmax[i][1], xmin=0, xmax=len(df), colors='r')
```

- Loops through each cluster's minimum and maximum values (`minmax`).
- Plots horizontal lines (`hlines`) across the entire plot:
- A green line at the minimum value of the cluster (`minmax[i][0]`).
- A red line at the maximum value of the cluster (`minmax[i][1]`).

Conclusion

The K-means clustering algorithm offers an effective approach to identifying support and resistance levels in financial markets. However, the success of this approach is closely related to proper feature selection, determining an appropriate number of clusters, and careful implementation of the algorithm. Combining the K-means algorithm with other technical analysis tools and indicators can also help obtain more reliable support and resistance levels.

Below, you can find the complete set of codes:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.cluster import KMeans
from kneed import DataGenerator, KneeLocator
import matplotlib.pyplot as plt

# Fetching stock data
ticker = 'GOOG' # Ticker symbol of the stock to trade
start_date = '2018-03-22' # Start date for fetching the data

df = yf.download(
    tickers=ticker,
    start=start_date,
    progress=False
)[['Close']] # Selecting only the 'Close' column

# Finding the optimal number of clusters
sum_of_squared_distances = []
K = range(1, 15)
for k in K:
    km = KMeans(n_clusters=k)
    km = km.fit(df.values.reshape(-1, 1))
    sum_of_squared_distances.append(km.inertia_)
kn = KneeLocator(K, sum_of_squared_distances, S=1.0, curve="convex", direction="down")

# Applying K-means clustering
kmeans = KMeans(n_clusters=kn.knee).fit(df.values.reshape(-1, 1))
c = kmeans.predict(df.values.reshape(-1, 1))

# Calculating min-max values for each cluster
minmax = []
for i in range(kn.knee):
    minmax.append([-np.inf, np.inf])
for i in range(len(df)):
    cluster = c[i]
    if df.values[i] > minmax[cluster][0]:
        minmax[cluster][0] = df.values[i]
    if df.values[i] < minmax[cluster][1]:
        minmax[cluster][1] = df.values[i]

# Plotting
plt.figure(figsize=(10, 6))
```

```
# Scatter plot of data points with cluster colors
for i in range(len(df)):
    colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
    color = colors[c[i]]
    plt.scatter(i, df.values[i], c=color, s=1)

# Plotting horizontal lines for min-max values of each cluster
for i in range(len(minmax)):
    plt.hlines(minmax[i][0], xmin=0, xmax=len(df), colors='g')
    plt.hlines(minmax[i][1], xmin=0, xmax=len(df), colors='r')

plt.xlabel('Data Point Index')
plt.ylabel('Close Price')
plt.title('Scatter Plot with Cluster Boundaries')
plt.show()
```



Written by Cemal Öztürk, Ph.D.

51 followers · 39 following

Follow

Ph.D. in Economics | Computer Engineer | Mathematician | Academic at Pamukkale University | Passionate about research, technology, and interdisciplinary studies

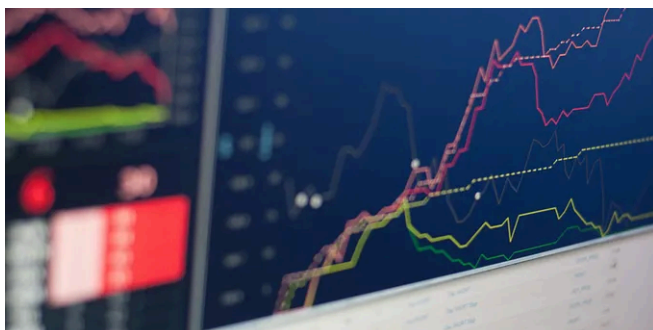
No responses yet



Write a response

What are your thoughts?

More from Cemal Öztürk, Ph.D.



 Cemal Öztürk, Ph.D.

Unveiling Cointegration: Johansen Test Explained with Python...

Financial time series analysis is a crucial aspect of economics and finance....

May 17, 2024  5  3



 Cemal Öztürk, Ph.D.

A Markov Regime Switching Approach to Characterizing...

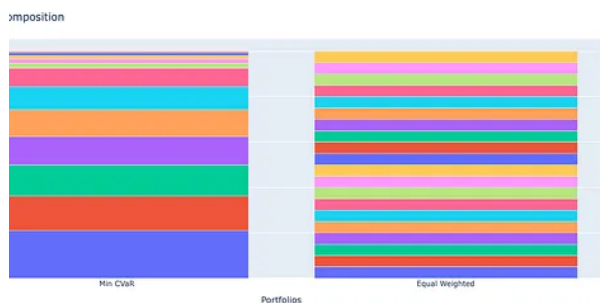
Unveiling market dynamics with Markov Regime Switching: Analyze volatility, regime...

Feb 3  34



 Cemal Öztürk, Ph.D.

Exploring Financial Volatility with GARCH Model: A Step-by-Step...



 Cemal Öztürk, Ph.D.

Mastering Portfolio Optimization: A Comprehensive Guide with Python

Introduction

Jun 18, 2024



9



2



Jun 24, 2024



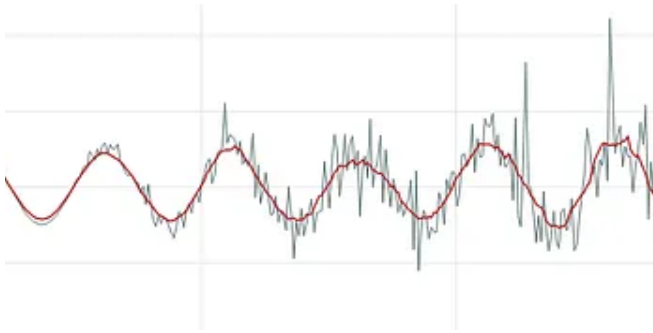
78



1

[See all from Cemal Öztürk, Ph.D.](#)

Recommended from Medium



Dmitriy Bolotov

Six Approaches to Time Series Smoothing

A practical overview of common smoothing techniques

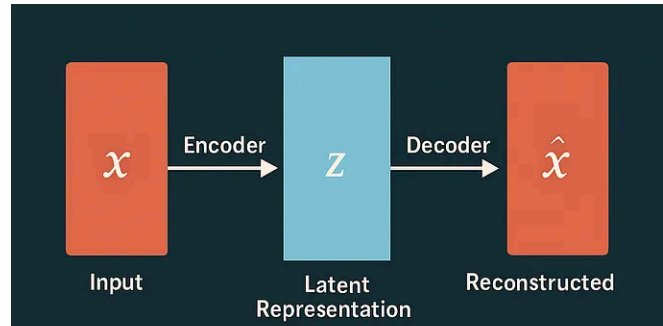
Jun 29



89



2



Roshmita Dey

Autoencoders for Anomaly Detection

An autoencoder is a type of neural network designed to learn an identity mapping under...

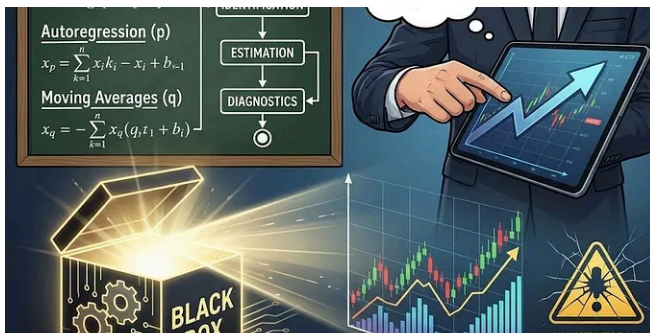


Dec 7



19



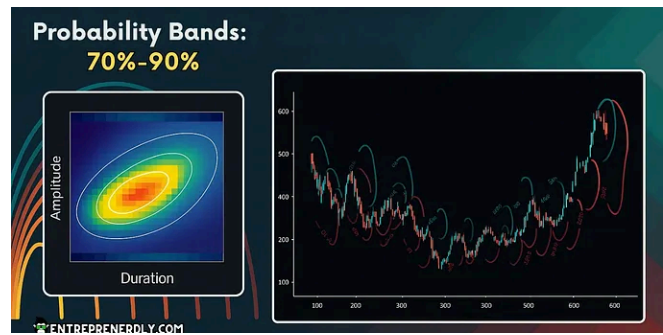


Simplifiedzone

“Where is the stock going next week?": Forecasting, Mechanics,...

Originally published at <https://simplified-zone.com> on December 11, 2025.

★ Dec 11 🖱 13 💬 1

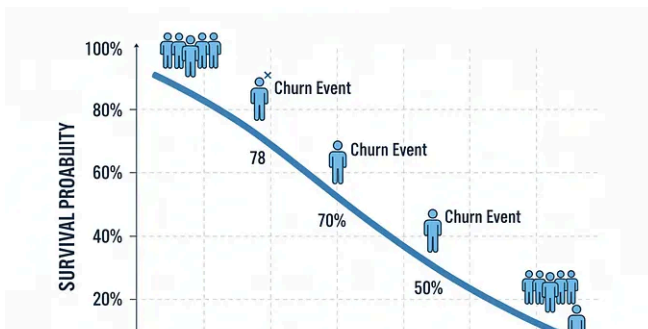


Cristian Velasquez

Visualizing Reversal Probability Zones

Implement Statistical Reversal Bands By Detecting Regimes, Quantifying Swing...

★ 3d ago 🖱 77 💬 2



Stefan Yankov

Introduction to Customer Survival Analysis: Understanding Custom...

Part 1.1 of the “Unlocking Customer Insights” series Explore the full series here: [Unlocking...](#)

★ Sep 9 🖱 2



$$Y_t = T_t + S_t + R_t$$

Where:

- Y_t is the observed value at time t
- T_t is the trend component at time t
- S_t is the seasonal component at time t
- R_t is the residual or irregular component at time t

Saurav

Chapter 1: Understanding time series forecasting

Time series forecasting is a specialized area of data analysis and machine learning...

Jul 22 🖱 1



See more recommendations